

Lecture 6

Regularization, Dimension Reduction, and High Dimensional Data

6.1

Contents

1 Shrinkage Estimation Methods	1
1.1 Ridge Regression	2
1.2 The Lasso	5
1.3 Elastic Nets	6
1.4 Another Formulation for Ridge Regression, the Lasso and Best Subset Selection	6
1.5 Graphical Visualization of Solutions	7
1.6 Selecting the Tuning Parameter λ	9
1.7 Example	9
2 Dimension Reduction Methods	20
2.1 Principal Components Regression	21
3 High Dimensional Data	27
3.1 Interpreting Results in High Dimensions	30

Some figures in this presentation are taken from 'An Introduction to Statistical Learning, with applications in R' (Springer, 2013) with permission from the authors: G. James, D. Witten, T. Hastie and R. Tibshirani.

Most of the material is extracted from the slides of the MOOC course by Hastie and Tibshirani, available at <https://www.r-bloggers.com/in-depth-introduction-to-machine-learning-in-15-hours-of-expert-videos/>.

6.2

1 Shrinkage Estimation Methods

These methods are used to estimate a linear model

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon$$

by performing a *regularization* or *shrinking* of parameter estimates *towards zero*.

Advantages

- shrinking the coefficient estimates can significantly reduce their variance
- it improves coefficients interpretability

Methods

- Ridge regression
- Lasso

6.3

1.1 Ridge Regression

Ordinary Least Squares estimates of parameters in a linear model are obtained as

$$\hat{\beta} = \arg \min_{\beta} RSS_{training} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\}.$$

Parameter estimates according to the ridge regression method are given by

$$\begin{aligned} \hat{\beta}_{\lambda}^R &= \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\} = \\ &= \arg \min_{\beta} \left\{ RSS_{training} + \lambda \sum_{j=1}^p \beta_j^2 \right\}, \end{aligned}$$

where $\lambda \geq 0$ is a tuning parameter.

6.4

Ridge regression estimate involves the simultaneous minimization of the two following components of the criterion function:

- $RSS_{training}$
- $\lambda \sum_{j=1}^p \beta_j^2 = \lambda (\|\beta\|_2)^2$

where $\|\beta\|_2$ is the norm of the vector $\beta = [\beta_1, \dots, \beta_p]$ in ℓ^2 .

6.5

The quantity $\lambda \sum_{j=1}^p \beta_j^2$ is said shrinkage penalty, and is function only of the coefficients $\beta_1, \dots, \beta_p$; it is small when $\beta_1, \dots, \beta_p$ are close to zero, and so it has the effect of shrinking the estimates of β_j towards zero.

Remarks

The estimate $\hat{\beta}_{\lambda}^R$ depends on the tuning parameter λ , to be determined separately.

- With $\lambda = 0$ the shrinkage penalty has no effect and $\hat{\beta}_0^R = \hat{\beta}_{OLS}$.
- When $\lambda \rightarrow \infty$ we have $\hat{\beta}_{\lambda 1}^R \rightarrow 0, \dots, \hat{\beta}_{\lambda p}^R \rightarrow 0$.

6.6

Remarks

The estimator $\hat{\beta}_{\lambda}^R$ is

$$\hat{\beta}_{\lambda}^R = (\mathbf{X}'\mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}'\mathbf{Y}$$

and we have

$$E \{ \hat{\beta}_{\lambda}^R \} = (\mathbf{X}'\mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}'\mathbf{X}\beta.$$

Since $\hat{\beta}_{OLS}$ is an unbiased estimator, we have that $\hat{\beta}_{\lambda}^R$ is an unbiased estimator of β only when $\lambda = 0$.

6.7

Remarks

Ridge regression has to be applied on standardized predictors:

- a transformation of the type $X_j \rightarrow cX_j$ will influence only the OLS estimate of $\beta_j \rightarrow \frac{1}{c}\beta_j$ and the model does not change; namely, we have

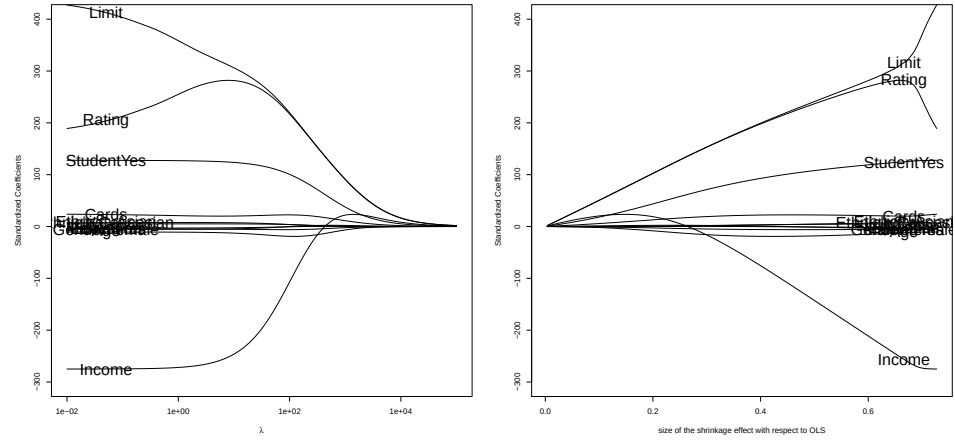
$$X_j \beta_j = cX_j \frac{1}{c}\beta_j$$

- with reference to the ridge regression estimates, a transformation of the type $X_j \rightarrow cX_j$ will affect estimates of all coefficients $\beta_1, \dots, \beta_p$.

Therefore if predictor standardization is not performed, the parameter estimates would depend on measurement scales of the predictors.

The following figure report the ridge regression estimates for the credit data set

6.8



6.9

The left side shows estimates as a function of the λ tuning parameter.

The right side shows estimates as a function of the entity of the shrinkage effect, having as reference the OLS estimate

$$\frac{\|\hat{\beta}_{\lambda}^R\|_2}{\|\hat{\beta}_{OLS}\|_2}$$

- When $\lambda = 0$ we have $\frac{\|\hat{\beta}_{\lambda}^R\|_2}{\|\hat{\beta}_{OLS}\|_2} = 1$ that is no shrinkage effect
- When $\lambda \rightarrow \infty$ we have $\frac{\|\hat{\beta}_{\lambda}^R\|_2}{\|\hat{\beta}_{OLS}\|_2} = 0$ that corresponds to the maximum shrinkage effect.

6.10

Advantages of ridge regression with respect to OLS (1)

Ridge regression's advantage over least squares is rooted in the *bias-variance trade-off*. As λ increases, the flexibility of the ridge regression fit decreases, leading to decreased variance but increased bias. We thus have:

- reduction of the variance of the estimate $\hat{f}$
- a bias increase

6.11

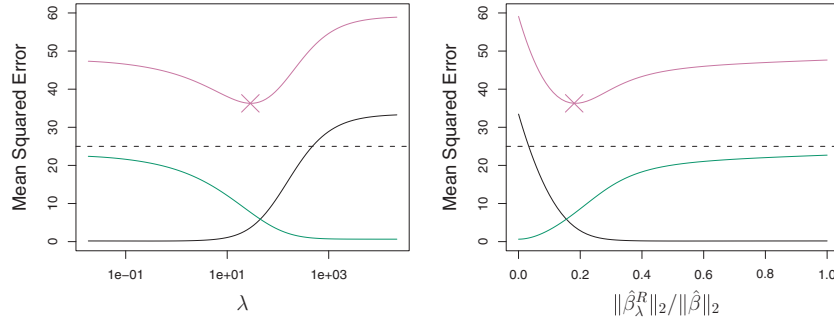


FIGURE 6.5. Squared bias (black), variance (green), and test mean squared error (purple) for the ridge regression predictions on a simulated data set, as a function of λ and $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$. The horizontal dashed lines indicate the minimum possible MSE. The purple crosses indicate the ridge regression models for which the MSE is smallest.

6.12

Therefore, we can expect that there exist ridge regression models with $\lambda > 0$ that have MSE_{test} lower than OLS; (OLS corresponds to $\lambda = 0$). We can minimize MSE_{test} with some optimum value of λ .

6.13

Advantages of ridge regression with respect to OLS (2)

When the sample size n is not much larger than the number of parameters p , ($n \not\gg p$), even if the real function f is close to linear, the OLS estimates will have low bias and high variability. Namely, we have

$$\hat{\Sigma}_{\hat{\beta}} = s^2 (\mathbf{X}'\mathbf{X})^{-1}$$

and

$$s^2 = \frac{1}{n - p - 1} RSS.$$

In these situations, ridge regression represents a good solution since it results in a lower variance than the OLS estimation method.

6.14

Comparison between ridge regression and best subset selection

- Best subset selection requires the estimate of 2^p models.
Ridge regression estimates only one model for each value of λ ; the iterative estimation algorithm is very efficient with execution times, for all values of λ , almost identical to those for fitting a linear model using least squares.
- Best subset selection selects a subset of variables.
The ridge regression model contains *all* variables.

6.15

1.2 The Lasso

Parameter estimates according to the lasso (Least Absolute Shrinkage and Selection Operator) method are given by

$$\begin{aligned}\hat{\beta}_{\lambda}^L &= \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\} = \\ &= \arg \min_{\beta} \left\{ RSS_{training} + \lambda \sum_{j=1}^p |\beta_j| \right\},\end{aligned}$$

where $\lambda \geq 0$ is a tuning parameter.

The shrinkage penalty $\lambda \sum_{j=1}^p |\beta_j| = \lambda \|\beta\|_1$ is always a function only of the coefficients $\beta_1, \dots, \beta_p$ and has a low value when the estimates of $\beta_1, \dots, \beta_p$ are close to 0. $\|\beta\|_1$ is the norm of $\beta = [\beta_1, \dots, \beta_p]$ in ℓ^1 .

Some of the coefficients estimated with the lasso can be exactly equal to zero.

Hence, much like best subset selection, the lasso performs *variable selection*.

Remark

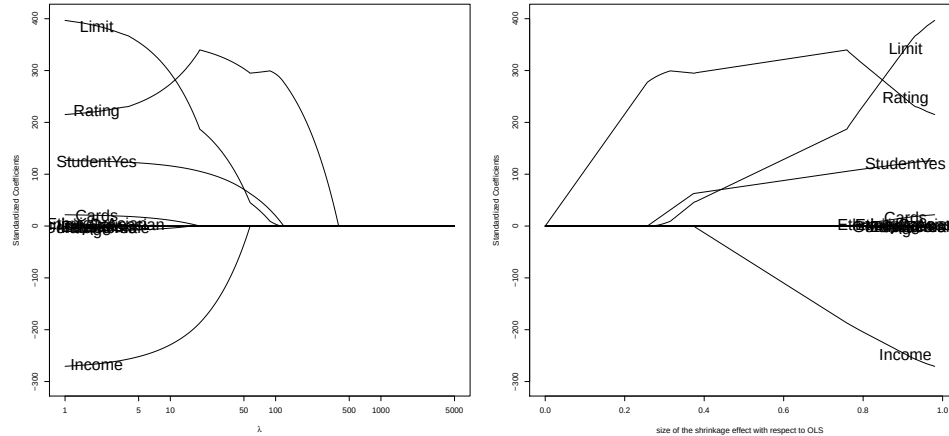
As with ridge regression, the lasso estimate is done on standardized *predictors*.

Remark

As with ridge regression we have a shrinkage effect on parameter estimates.

A sufficiently large value of λ has the effect of forcing some of the coefficient estimates to be exactly equal to zero, thus obtaining a so-called sparse model, that includes only a subset of predictors. Therefore the lasso performs *variable selection* and allows a better interpretability of results.

The following figure report the lasso estimates for the credit data set



The left side shows estimates as a function of the λ tuning parameter.

The right side shows estimates as a function of the entity of the shrinkage effect, having as reference the OLS estimate

$$\frac{\|\hat{\beta}_{\lambda}^L\|_1}{\|\hat{\beta}_{OLS}\|_1}$$

- When $\lambda = 0$ the shrinkage penalty has no effect on estimates and $\hat{\beta}_{\lambda}^L = \hat{\beta}_{OLS}$.
- When $\lambda \rightarrow \infty$ we have $\hat{\beta}_{\lambda 1}^L = \dots = \hat{\beta}_{\lambda p}^L = 0$.

The number of variables in the model depends negatively on the size of the tuning parameter λ .

6.20

1.3 Elastic Nets

Elastic Net is a class of estimation methods that includes the ridge regression and the lasso.

Parameter estimates according to Elastic Net are given by

$$\hat{\beta}_{\lambda}^{EN} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \left[\frac{1}{2} (1 - \alpha) (\|\beta\|_2)^2 + \alpha \|\beta\|_1 \right] \right\}$$

where:

- $\lambda \geq 0$ is a tuning parameter
- α is a mixing parameter:
 - with $\alpha = 0$ we have the ridge regression,
 - with $\alpha = 1$ we have the lasso.

6.21

1.4 Another Formulation for Ridge Regression, the Lasso and Best Subset Selection

Ridge regression, the lasso and best subset selection solve the optimization problem

$$\min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\}$$

subject to the following constraints

- Ridge regression

$$\sum_{j=1}^p \beta_j^2 \leq s$$

- Lasso

$$\sum_{j=1}^p |\beta_j| \leq s$$

- Best subset selection

$$\sum_{j=1}^p I(\beta_j \neq 0) \leq s$$

Best subset selection requires to the algorithm to include a number of predictors not larger than s .

6.22

With reference to ridge regression and the lasso, if $s \rightarrow +\infty$ we have $\lambda \rightarrow 0$ and the estimation methods are equivalent to OLS, that does not impose any constraint to the β_j parameters.

6.23

1.5 Graphical Visualization of Solutions

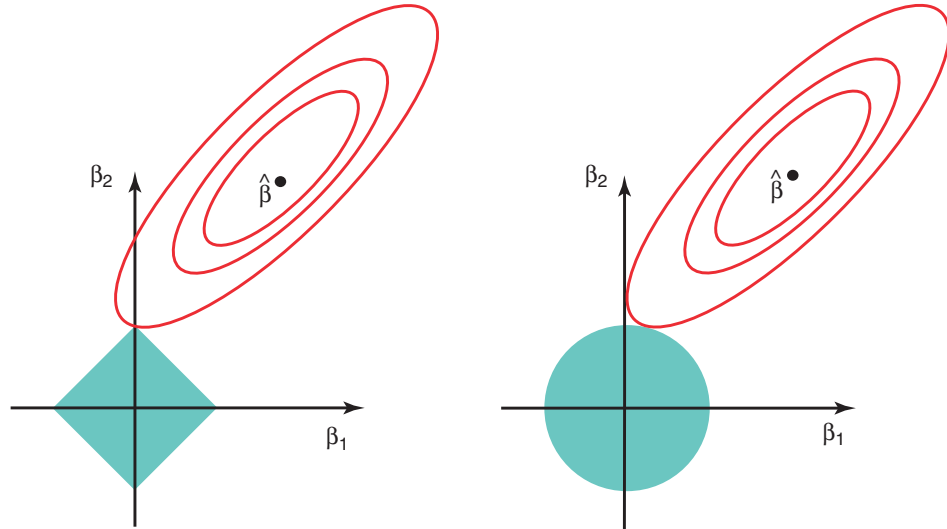


FIGURE 6.7. Contours of the error and constraint functions for the lasso (left) and ridge regression (right). The solid blue areas are the constraint regions, $|\beta_1| + |\beta_2| \leq s$ and $\beta_1^2 + \beta_2^2 \leq s$, while the red ellipses are the contours of the RSS.

6.24

Let us denote with $\hat{\beta}$ the OLS solution.

The search of the minimum is made in the grey coloured sets (an hypersphere with ridge regression and an hypercube with the lasso).

The λ tuning parameter establishes the size of the regions.

If $\hat{\beta}_{OLS}$ belongs to the regions given by the constraints, then $\hat{\beta}_\lambda^R = \hat{\beta}_{OLS}$ or $\hat{\beta}_\lambda^L = \hat{\beta}_{OLS}$.

Otherwise the solutions are given by the first point at which an ellipse contacts the constraint region.

Solution is unique.

6.25

With ridge regression a coefficient is set exactly to 0 *only* if the contact point is on one of the two axes.

With the lasso we have a larger probability that the solution corresponds to zero coefficients, this happens whenever the solution is on one of the two axes.

6.26

Comparing the lasso and ridge Regression

The lasso has a major advantage over ridge regression, in that it produces simpler and more interpretable models that involve only a subset of the predictors.

We analyze the following figures to evaluate prediction accuracy

6.27

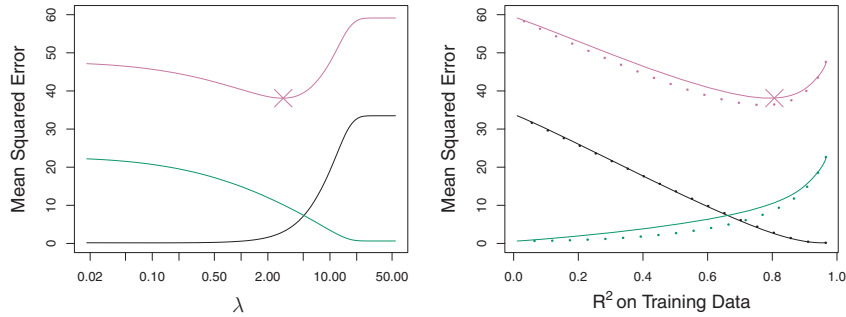


FIGURE 6.8. Left: Plots of squared bias (black), variance (green), and test MSE (purple) for the lasso on a simulated data set. Right: Comparison of squared bias, variance and test MSE between lasso (solid) and ridge (dashed). Both are plotted against their R^2 on the training data, as a common form of indexing. The crosses in both plots indicate the lasso model for which the MSE is smallest.

6.28

We have $p = 45$ predictors and $n = 50$ observations (the same data used in Figure 6.5 by JWHT). The response Y depends on all variables.

The left side of the figure shows that lasso behaves similarly to ridge regression: when λ increases we observe a reduction of the variance of the estimation method and an increase of the bias.

The left side of the figure compares the lasso and the ridge regression by analyzing the behaviour of the MSE as a function of the R^2 , the latter one evaluated on training data. The variance of ridge regression is slightly lower than that of the lasso. Remind that the lasso assumes that some coefficient is null.

6.29

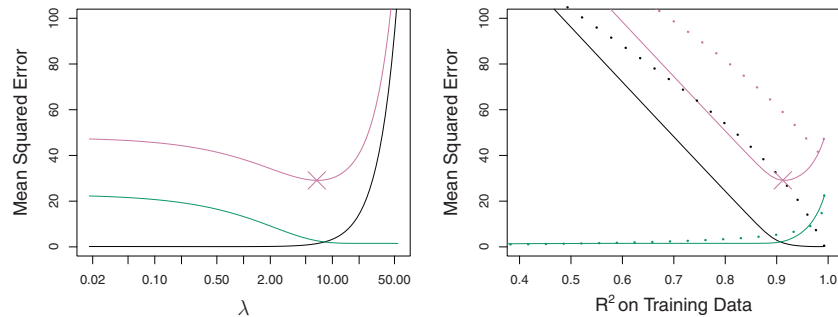


FIGURE 6.9. Left: Plots of squared bias (black), variance (green), and test MSE (purple) for the lasso. The simulated data is similar to that in Figure 6.8, except that now only two predictors are related to the response. Right: Comparison of squared bias, variance and test MSE between lasso (solid) and ridge (dashed). Both are plotted against their R^2 on the training data, as a common form of

6.30

We now consider a data set always with $p = 45$ predictors and $n = 50$ observations, but with a response Y that depends only on 2 predictors.

In this case the lasso performs better than ridge regression.

These two examples illustrate that neither ridge regression nor the lasso will universally dominate the other.

6.31

- In general, one might expect the lasso to perform better in a setting where a relatively small number of predictors have substantial coefficients, and the remaining predictors have coefficients that are very small or that equal zero.

- Ridge regression will perform better when the response is a function of many predictors, all with coefficients of roughly equal size.

6.32

However, the number of predictors that is related to the response is never known a priori for real data sets. A technique such as cross-validation can be used in order to determine which approach is better on a particular data set.

As with ridge regression, when the least squares estimates have excessively high variance (e.g. when it does not hold that $n \gg p$), the lasso solution can yield a reduction in variance at the expense of a small increase in bias, and consequently can generate more accurate predictions.

Unlike ridge regression, the lasso performs variable selection, and hence results in models that are easier to interpret.

6.33

1.6 Selecting the Tuning Parameter λ

We can use cross validation techniques

1. define a grid for λ
2. compute the cross validation error for each value of λ and select the parameter value for which the cross validation error is smallest
3. finally, the model is re-fit using all of the available observations and the selected value of the tuning parameter.

6.34

1.7 Example

```
# data reading
credit <- read.table("Credit.csv", sep = ",", header = T)
names(credit)
## [1] "X"          "Income"     "Limit"      "Rating"
## [5] "Cards"     "Age"        "Education"  "Gender"
## [9] "Student"   "Married"    "Ethnicity"  "Balance"
# the first variable contains case names
credit <- credit[, -1]
# number of subjects
(n <- nrow(credit))
## [1] 400
# extraction of the model.matrix for the complete regression
# (the first column referring to the intercept has to be
# dropped)
x <- model.matrix(Balance ~ ., data = credit)[, -1]
# response
y <- credit$Balance
```

6.35

```
# Ridge Regression
library(glmnet)
## Loading required package: Matrix
## Loading required package: foreach
## Loaded glmnet 2.0-18
# grid for the tuning parameter \lambda
grid <- c(10^seq(from = 10, to = -2, length = 100), 0)
# glmnet function: argument alpha=0 --> ridge, alpha=1 -->
# lasso
```

```

ridge.mod <- glmnet(x, y, alpha = 0, lambda = grid)
# predictors are by default standardized (standardize=TRUE)
head(ridge.mod$lambda)
## [1] 10000000000 7564633276 5722367659 4328761281
## [5] 3274549163 2477076356
tail(ridge.mod$lambda)
## [1] 0.03053856 0.02310130 0.01747528 0.01321941 0.01000000
## [6] 0.00000000
dim(coef(ridge.mod))
## [1] 12 101

```

6.36

```

# coefficients for lambda at position 50 in the grid array
grid[50]
## [1] 11497.57
ridge.mod$lambda[50]
## [1] 11497.57
coef(ridge.mod)[, 50]
##      (Intercept)      Income      Limit
##      443.882328225      0.207297549      0.006255511
##      Rating      Cards      Age
##      0.093529429      1.094782827      -0.007423108
##      Education      GenderFemale      StudentYes
##      -0.036491175      0.727221775      15.224286108
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      -0.273927934      -0.323636288      -0.088984490
# l_2 norm (intercept is excluded)
sqrt(sum(coef(ridge.mod)[-1, 50]^2))
## [1] 15.28879

```

6.37

```

# coefficients for lambda at position 60 in the grid array
ridge.mod$lambda[60]
## [1] 705.4802
coef(ridge.mod)[, 60]
##      (Intercept)      Income      Limit
##      11.21325341      0.47047426      0.04709688
##      Rating      Cards      Age
##      0.70328171      10.00746444      -0.54061006
##      Education      GenderFemale      StudentYes
##      0.01398463      4.61657214      156.68825813
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      -6.12237541      0.62039885      1.43859125
# l_2 norm (intercept is excluded)
sqrt(sum(coef(ridge.mod)[-1, 60]^2))
## [1] 157.2057

```

6.38

```

# coefficients for lambda at position 101 in the grid array
ridge.mod$lambda[101]
## [1] 0
coef(ridge.mod)[, 101]

```

```
##      (Intercept)      Income      Limit
##      -484.4664275      -7.8008940      0.1765862
##      Rating      Cards      Age
##      1.3497369      16.6924821      -0.6160067
##      Education      GenderFemale      StudentYes
##      -1.0448695      -10.6573380      425.0483299
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      -9.0274912      17.1763212      10.1479251
# l_2 norm (intercept is excluded)
sqrt(sum(coef(ridge.mod)[-1, 101]^2))
## [1] 426.1478
```

6.39

```
# OLS estimate
coef(lm(y ~ x))
##      (Intercept)      xIncome      xLimit
##      -479.2078706      -7.8031018      0.1909067
##      xRating      xCards      xAge
##      1.1365265      17.7244836      -0.6139088
##      xEducation      xGenderFemale      xStudentYes
##      -1.0988553      -10.6532477      425.7473595
##      xMarriedYes      xEthnicityAsian      xEthnicityCaucasian
##      -8.5339006      16.8041792      10.1070252
# the precision parameter of the iterative algorithm has to
# be increased in order to obtain more precise values
coef(glmnet(x, y, alpha = 0, lambda = grid, thresh = 1e-25))[,
101]
##      (Intercept)      Income      Limit
##      -479.2078706      -7.8031018      0.1909067
##      Rating      Cards      Age
##      1.1365265      17.7244836      -0.6139088
##      Education      GenderFemale      StudentYes
##      -1.0988553      -10.6532477      425.7473595
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      -8.5339006      16.8041792      10.1070252
```

6.40

```
# coefficients for lambda=10^10 at position 1 in the grid
# array
ridge.mod$lambda[1]
## [1] 1e+10
coef(ridge.mod)[, 1]
##      (Intercept)      Income      Limit
##      5.200149e+02      2.777310e-07      7.881306e-09
##      Rating      Cards      Age
##      1.178376e-07      1.331034e-06      2.245918e-09
##      Education      GenderFemale      StudentYes
##      -5.445751e-08      9.061130e-07      1.820460e-05
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      -2.455471e-07      -4.746711e-07      -1.386700e-07
# l_2 norm (intercept is excluded)
sqrt(sum(coef(ridge.mod)[-1, 1]^2))
## [1] 1.828658e-05
```

```
# estimate OLS model costante
```

```
coef(lm(y ~ 1))
```

```
## (Intercept)
```

```
##      520.015
```

```
mean(y)
```

```
## [1] 520.015
```

6.41

```
# prediction (s is a new value of lambda)
```

```
predict(ridge.mod, s = 50, type = "coefficients")
```

```
## 12 x 1 sparse Matrix of class "dgCMatrix"
```

```
##              1
```

```
## (Intercept)    -381.7320456
```

```
## Income         -4.7079670
```

```
## Limit          0.1102715
```

```
## Rating         1.6083591
```

```
## Cards          16.0197690
```

```
## Age            -0.9992703
```

```
## Education      -0.4049496
```

```
## GenderFemale   -3.8463610
```

```
## StudentYes     372.6729031
```

```
## MarriedYes     -12.2654743
```

```
## EthnicityAsian  12.1971765
```

```
## EthnicityCaucasian 8.8115489
```

6.42

```
# estimation of MSE_test
```

```
# definition of indices to select the training data set
```

```
set.seed(1234)
```

```
train <- sample(n, n/2)
```

```
y.test <- y[-train]
```

```
# ridge regression estimate on the training data set
```

```
ridge.mod <- glmnet(x[train, ], y[train], alpha = 0, lambda = grid,  
  thresh = 1e-25)
```

```
# compute predictions corresponding to lambda = 4 newx matrix  
# with test data set
```

```
ridge.pred <- predict(ridge.mod, newx = x[-train, ], s = 4)
```

```
# estimation of the MSE_test
```

```
mean((y.test - ridge.pred)^2)
```

```
## [1] 10182.9
```

6.43

```
# in order to find the best value for lambda (corresponding
```

```
# to minimum MSE_test) we can perform a cross validation
```

```
# (cv.glmnet function) by default a 10-fold cv
```

```
RNGkind(sample.kind = "Rounding")
```

```
# we used the old \texttt{sample.kind = 'Rounding'} method
```

```
# for generating sample in order to ensure reproducibility
```

```
# with JWHT results
```

```
set.seed(1234)
```

```
train <- sample(n, n/2)
```

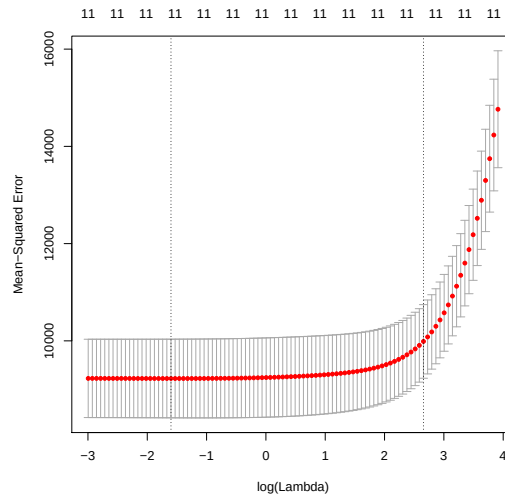
```
cv.out <- cv.glmnet(x[train, ], y[train], lambda = 5 * 10^seq(-2,  
  1, length = 100), alpha = 0)
```

```
# plot estimated 10fold cv: MSE_test as a function of
```

```
# log(lambda)
plot(cv.out)
```

6.44

```
## Warning in RNGkind(sample.kind = "Rounding"): non-uniform
'Rounding' sampler used
```



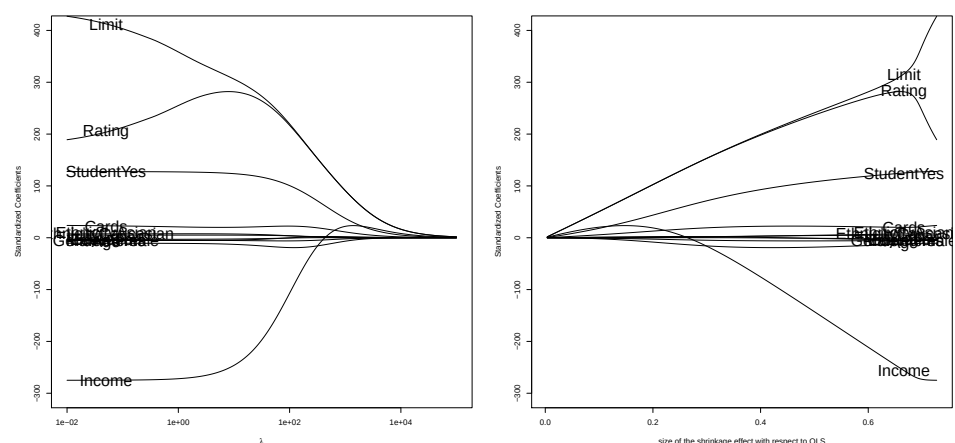
6.45

```
# best lambda
(best.lambda <- cv.out$lambda.min)
## [1] 0.2018509
# estimate of MSE_test with best.lambda
ridge.pred <- predict(ridge.mod, newx = x[-train, ], s = best.lambda)
mean((y.test - ridge.pred)^2)
## [1] 372562.5
# re-fit the model on all available data
ridge.mod <- glmnet(x, y, alpha = 0)
predict(ridge.mod, s = best.lambda, type = "coefficients")[1:12,
]
##      (Intercept)      Income      Limit
##    -400.8600617    -5.1751854     0.1144898
##      Rating      Cards      Age
##     1.6607396    15.8061272    -0.9565674
##    Education GenderFemale StudentYes
##    -0.4740526    -4.8551984    381.6895970
##   MarriedYes EthnicityAsian EthnicityCaucasian
##   -12.0976162    13.1298333     9.1146561
# no coefficient has been set to zero; the coefficient of
# Limit is lower than that of the collinear variable Rating
# Comparison with OLS
lm(y ~ x)
##
## Call:
## lm(formula = y ~ x)
##
## Coefficients:
```

```
##          (Intercept)                xIncome
##          -479.2079                -7.8031
##          xLimit                    xRating
##          0.1909                    1.1365
##          xCards                    xAge
##          17.7245                    -0.6139
##          xEducation                xGenderFemale
##          -1.0989                    -10.6532
##          xStudentYes                xMarriedYes
##          425.7474                    -8.5339
##          xEthnicityAsian  xEthnicityCaucasian
##          16.8042                    10.1070
```

6.46

```
# plot parameter estimates as a function of \lambda semistd
# coefs
layout(matrix(1:2, 1, 2))
x1 <- scale(x)
grid <- c(10^seq(from = 5, to = -2, length = 1000))
ridge.mod <- glmnet(x1, y, alpha = 0, lambda = grid)
coef.lambda <- sapply(1:length(grid), function(i) coef(ridge.mod)[,
i])
coef.lambda <- t(coef.lambda[-1, ])
plot(1, 1, type = "n", xlim = range(grid), ylim = c(-300, 400),
     log = "x", xlab = expression(lambda), ylab = "Standardized Coefficients")
sapply(1:11, function(i) lines(grid, coef.lambda[, i]))
sapply(1:11, function(i) text(grid[1000 - 100], coef.lambda[1000 -
100, i], colnames(x1)[i], cex = 2))
l2.norm <- function(x) sqrt(sum(x^2))
grid1 <- apply(coef.lambda, 1, l2.norm)/l2.norm(coef(lm(y ~ scale(x1))))
plot(0, 0, type = "n", xlim = range(grid1), ylim = c(-300, 400),
     xlab = "size of the shrinkage effect with respect to OLS",
     ylab = "Standardized Coefficients")
sapply(1:11, function(i) lines(grid1, coef.lambda[, i]))
sapply(1:11, function(i) text(grid1[1000 - 400], coef.lambda[1000 -
400, i], colnames(x1)[i], cex = 2))
```



6.47

```
# Lasso grid for the tuning parameter \lambda
grid <- c(10^seq(from = 10, to = -2, length = 100), 0)
# glmnet function: argument alpha=0 --> ridge, alpha=1 -->
```

```
# lasso
lasso.mod <- glmnet(x, y, alpha = 1, lambda = grid)
# predictors are by default standardized (standardize=TRUE)
dim(coef(lasso.mod))
## [1] 12 101
# coefficients for lambda at position 79 in the grid array
grid[79]
## [1] 3.511192
coef(lasso.mod)[, 79]
##      (Intercept)      Income      Limit
##      -486.4599401      -7.3367372      0.1616428
##      Rating      Cards      Age
##      1.4632579      13.9974485      -0.4898250
##      Education      GenderFemale      StudentYes
##      0.0000000      -2.4872004      411.3928020
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      -1.6029368      1.5252144      0.0000000
# l_1 norm (intercept is excluded)
sum(abs(coef(lasso.mod)[-1, 79]))
## [1] 440.4571
```

6.48

```
# coefficients for lambda=0 at position 101 in the grid array
lasso.mod$lambda[101]
## [1] 0
coef(lasso.mod)[, 101]
##      (Intercept)      Income      Limit
##      -483.3390087      -7.7998397      0.1795386
##      Rating      Cards      Age
##      1.3055047      16.9077140      -0.6159204
##      Education      GenderFemale      StudentYes
##      -1.0558078      -10.6535086      425.1901150
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      -8.9241744      17.0850540      10.1341319
# l_1 norm (intercept is excluded)
sum(abs(coef(lasso.mod)[-1, 101]))
## [1] 499.8513
# OLS estimate
coef(lm(y ~ x))
##      (Intercept)      xIncome      xLimit
##      -479.2078706      -7.8031018      0.1909067
##      xRating      xCards      xAge
##      1.1365265      17.7244836      -0.6139088
##      xEducation      xGenderFemale      xStudentYes
##      -1.0988553      -10.6532477      425.7473595
##      xMarriedYes      xEthnicityAsian      xEthnicityCaucasian
##      -8.5339006      16.8041792      10.1070252
# the precision parameter of the iterative algorithm has to
# be increased in order to obtain more precise values
coef(glmnet(x, y, alpha = 1, lambda = grid, thresh = 1e-19))[,
101]
```

```
##      (Intercept)      Income      Limit
##      -479.2078759      -7.8031018      0.1909067
##      Rating      Cards      Age
##      1.1365267      17.7244826      -0.6139088
##      Education      GenderFemale      StudentYes
##      -1.0988553      -10.6532477      425.7473588
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      -8.5339011      16.8041795      10.1070252
```

6.49

```
# coefficients for lambda at position 40 in the grid array
lasso.mod$lambda[40]
## [1] 187381.7
coef(lasso.mod)[, 40]
##      (Intercept)      Income      Limit
##      520.015      0.000      0.000
##      Rating      Cards      Age
##      0.000      0.000      0.000
##      Education      GenderFemale      StudentYes
##      0.000      0.000      0.000
##      MarriedYes      EthnicityAsian      EthnicityCaucasian
##      0.000      0.000      0.000
# l_1 norm (intercept is excluded)
sum(abs(coef(lasso.mod)[-1, 1]))
## [1] 0
# OLS estimate (model with only the intercept)
coef(lm(y ~ 1))
## (Intercept)
##      520.015
mean(y)
## [1] 520.015
```

6.50

```
# estimation of MSE_test

# definition of indices to select the training data set

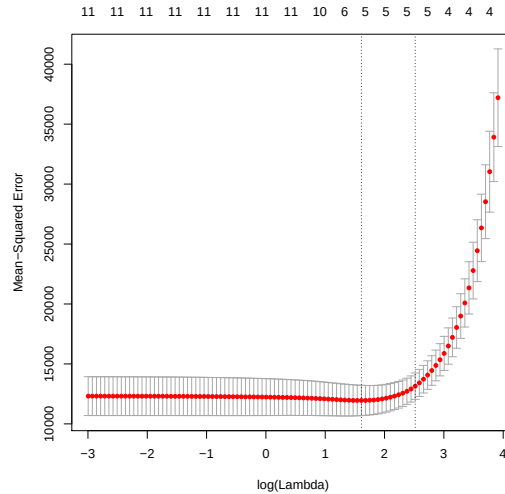
RNGkind(sample.kind = "Rounding")
# we used the old \texttt{sample.kind = 'Rounding'} method
# for generating sample in order to ensure reproducibility
# with JWHT results
set.seed(1400)
train <- sample(n, n/2)
y.test <- y[-train]
# Lasso estimate on the training data set
lasso.mod <- glmnet(x[train, ], y[train], alpha = 1, lambda = grid)
```

```
# in order to find the best value for lambda (corresponding
# to minimum MSE_test) we can perform a cross validation
# (cv.glmnet function) by default a 10-fold cv
cv.out <- cv.glmnet(x[train, ], y[train], lambda = 5 * 10^seq(-2,
  1, length = 100), alpha = 1)
# plot estimated 10fold cv: MSE_test as a function of
```

6.51


```
# log(lambda)
plot(cv.out)
```

6.52



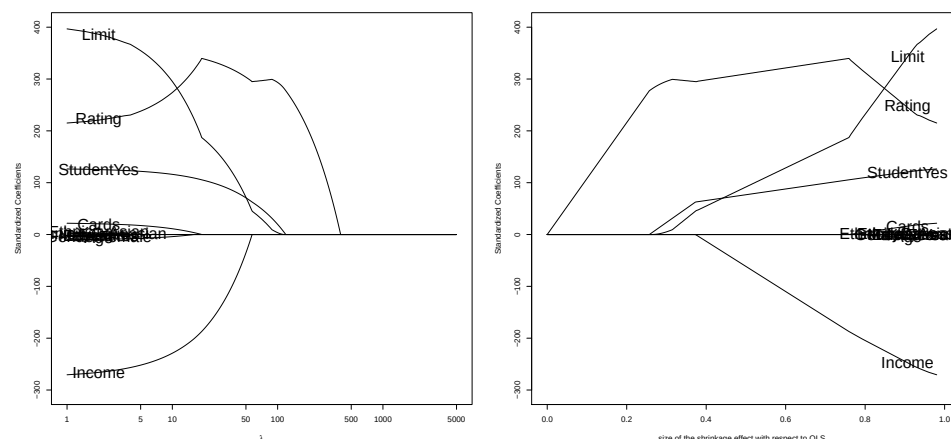
6.53

```
# best lambda
(best.lambda <- cv.out$lambda.min)
## [1] 5
# estimate of MSE_test with best.lambda
lasso.pred <- predict(lasso.mod, newx = x[-train, ], s = best.lambda)
mean((y.test - lasso.pred)^2)
## [1] 10167.56
# re-fit the model on all available data
lasso.mod <- glmnet(x, y, alpha = 1)
predict(lasso.mod, s = best.lambda, type = "coefficients")[1:12,
]
##           (Intercept)           Income           Limit
##      -4.841757e+02      -7.140800e+00      1.535960e-01
##           Rating           Cards           Age
##      1.537497e+00      1.265971e+01     -4.313240e-01
##           Education   GenderFemale   StudentYes
##      0.000000e+00     -9.126607e-04      4.057162e+02
##           MarriedYes   EthnicityAsian EthnicityCaucasian
##      0.000000e+00      0.000000e+00      0.000000e+00
# 5 coefficients have been set to zero; the coefficient of
# Limit is close to 0; Limit is quasi-collinear with Rating
# comparison with OLS
lm(y ~ x)
##
## Call:
## lm(formula = y ~ x)
##
## Coefficients:
##           (Intercept)           xIncome
##      -479.2079           -7.8031
##           xLimit           xRating
```

```
##          0.1909          1.1365
##          xCards          xAge
##          17.7245          -0.6139
##          xEducation      xGenderFemale
##          -1.0989          -10.6532
##          xStudentYes      xMarriedYes
##          425.7474          -8.5339
##          xEthnicityAsian  xEthnicityCaucasian
##          16.8042          10.1070
# ... try with a different set.seed ...
```

6.54

```
# plot parameter estimates as a function of \lambda semistd
# coefs
layout(matrix(1:2, 1, 2))
x1 <- scale(x)
grid <- c(seq(from = 5000, to = 1, length = 10000))
ridge.mod <- glmnet(x1, y, alpha = 1, lambda = grid)
coef.lambda <- sapply(1:length(grid), function(i) coef(ridge.mod[,
  i]))
coef.lambda <- t(coef.lambda[-1, ])
plot(1, 1, type = "n", xlim = range(grid), ylim = c(-300, 400),
  log = "x", xlab = expression(lambda), ylab = "Standardized Coefficients")
sapply(1:11, function(i) lines(grid, coef.lambda[, i]))
sapply(1:11, function(i) text(grid[10000 - 2], coef.lambda[10000 -
  2, i], colnames(x1)[i], cex = 2))
l1.norm <- function(x) sum(abs(x))
grid1 <- apply(coef.lambda, 1, l1.norm)/l1.norm(coef(lm(y ~ scale(x1)))[-1])
plot(0, 0, type = "n", xlim = range(grid1), ylim = c(-300, 400),
  xlab = "size of the shrinkage effect with respect to OLS",
  ylab = "Standardized Coefficients")
sapply(1:11, function(i) lines(grid1, coef.lambda[, i]))
sapply(1:11, function(i) text(grid1[10000 - 10], coef.lambda[10000 -
  10, i], colnames(x1)[i], cex = 2))
```



6.55

```
library(glmnet)
library(mvtnorm)

decomposition <- function(f, sigmaE, alpha) {
  varcov <- diag(45)
```

```

varcov[1, 2] <- varcov[2, 1] <- 0.99
varcov[2, 3] <- varcov[3, 2] <- 0.99
varcov[1, 3] <- varcov[3, 1] <- 0.99
varcov[5, 6] <- varcov[6, 5] <- 0.57
varcov[6, 7] <- varcov[7, 6] <- -0.06
varcov[5, 7] <- varcov[7, 5] <- 0.78
varcov[15, 16] <- varcov[16, 15] <- 0.57
varcov[16, 17] <- varcov[17, 16] <- -0.06
varcov[15, 17] <- varcov[17, 15] <- 0.78
varcov[25, 26] <- varcov[26, 25] <- 0.57
varcov[26, 27] <- varcov[27, 26] <- -0.06
varcov[25, 27] <- varcov[27, 25] <- 0.78
varcov[35, 36] <- varcov[36, 35] <- 0.57
varcov[36, 37] <- varcov[37, 36] <- -0.06
varcov[35, 37] <- varcov[37, 35] <- 0.78

set.seed(1234)
testxobs <- cbind(1, rmvnorm(ntest, sigma = varcov))
testyobs <- f(testxobs[, -1]) + rnorm(ntest, sd = sigmaE)
decompositionrr <- function(f) {
  x <- cbind(1, rmvnorm(ntrain, sigma = varcov))
  y <- f(x[, -1]) + rnorm(ntrain, sd = sigmaE)
  ridge.mod <- glmnet(x, y, alpha = alpha, lambda = grid)
  predict(ridge.mod, newx = testxobs, s = grid)
}
a <- replicate(1000, decompositionrr(f))
decomp <- rbind(apply((f(testxobs[, -1]) - apply(a, c(1,
  2), mean))^2, 2, mean), apply(apply(a, c(1, 2), var),
  2, mean))
plot(grid, decomp[1, ], type = "l", ylim = c(0, ceiling(max(decomp) +
  sigmaE^2)), xlab = "", ylab = "", bty = "n", col = "blue",
  log = "x")
lines(grid, decomp[2, ], col = "red")
lines(grid, sigmaE^2 + apply(decomp, 2, sum), lwd = 3)
abline(h = sigmaE^2, lty = 2)
}

```

6.56

```

p <- 45 #dispari
ntrain <- 50
ntest <- 100
f1 <- function(X) c(1 + X %*% betal)
f2 <- function(X) c(1 + X %*% beta2)
grid <- 10^seq(from = 4, to = -2, length = 100)

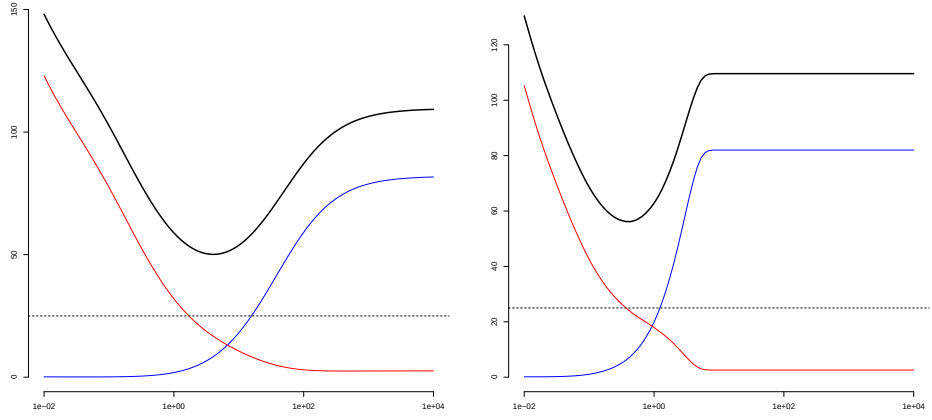
```

6.57

```

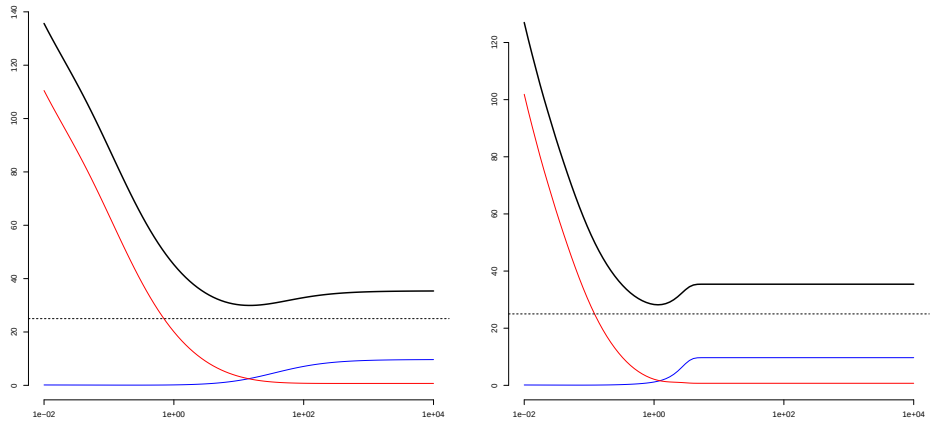
layout(matrix(1:2, 1, 2))
beta2 <- betal <- runif(p) * 2.5
betal[30:p] <- betal[30:p]/2
decomposition(f1, sigmaE = 5, alpha = 0) #ridge
decomposition(f1, sigmaE = 5, alpha = 1) #lasso

```



6.58

```
layout(matrix(1:2, 1, 2))
beta2 <- beta1 <- runif(p) * 2.5
beta2[3:p] <- 0
decomposition(f2, sigmaE = 5, alpha = 0) #ridge
decomposition(f2, sigmaE = 5, alpha = 1) #lasso
```



6.59

2 Dimension Reduction Methods

Let us consider a linear relationship referred to the generic subject

$$y_i = \beta_0 + \sum_{j=1}^p \beta_j x_{ij} + \varepsilon_i \quad (1)$$

and the linear transformation of the p predictors $X_1, \dots, X_p$ space in a space with dimension M , obtained by considering the $M \leq p$ linear combinations

$$Z_m = \sum_{j=1}^p \phi_{jm} X_j, \quad m = 1, \dots, M.$$

With $M < p$ we are performing a reduction of the dimension of the variable space.

6.60

Let

$$y_i = \theta_0 + \sum_{m=1}^M \theta_m z_{im} + \varepsilon_i \quad (2)$$

be the linear model that describes Y as a function of the new variables.

The new model can be formulated as a function of the predictors $X_1, \dots, X_p$ as follows

$$\begin{aligned} y_i &= \theta_0 + \sum_{m=1}^M \theta_m z_{im} + \varepsilon_i = \\ &= \theta_0 + \sum_{m=1}^M \theta_m \sum_{j=1}^p \phi_{jm} x_{ij} + \varepsilon_i = \\ &= \theta_0 + \sum_{j=1}^p \sum_{m=1}^M \theta_m \phi_{jm} x_{ij} + \varepsilon_i \end{aligned}$$

6.61

The model (2)

is a special case of the original regression model (1),

that has to be fit under the constraints

$$\beta_j = \sum_{m=1}^M \theta_m \phi_{jm}, \quad j = 1, 2, \dots, p.$$

6.62

Model (2) has a constrained solution; OLS applied to the original model (1) give an unconstrained solution.

This constraint on the form of the coefficients has the potential to bias the coefficient estimates. Observe that whenever $n \gg p$

- if $M \ll p$ can significantly reduce the variance of the fitted coefficients;
- if $M = p$ and all the Z_m are linearly independent, then no constraints is posed and we have the same estimates of model (1); we have only made a rotation of the original predictor space, a possible orthogonal change of basis.

6.63

The most commonly encountered reduction methods are:

- The Principal Components Analysis (unsupervised method)
- The Partial Least Squares (cfr. JWHT) (supervised method)

6.64

2.1 Principal Components Regression

Principal Components Regression Algorithm

The algorithm is made by the following steps

1. Find the principal components $Z_1, \dots, Z_M$
2. Fit the model using principal components as predictors

The M principal components are used as predictors of Y .

Assumption

It is assumed that a reduced number of principal components $Z_1, \dots, Z_M$ is sufficient to explain the variability present in predictors $X_1, \dots, X_p$ as well as the relationship of predictors with Y .

6.65

If the principal components $Z_1, \dots, Z_M$ describe exhaustively the variables $X_1, \dots, X_p$ then the estimate based on $M \ll p$ predictors can mitigate overfitting when $n \gg p$.

6.66

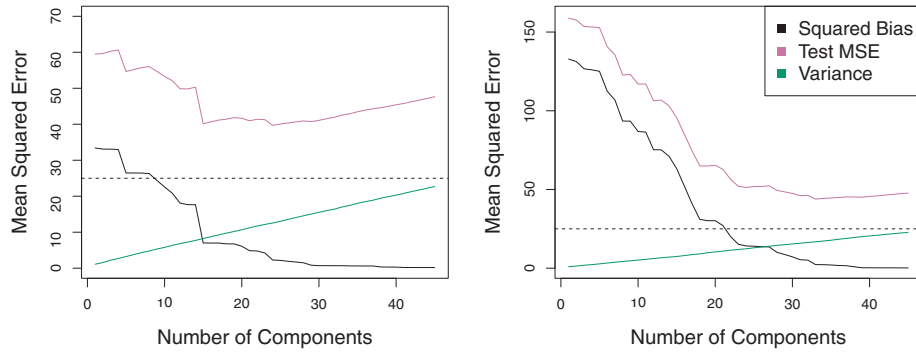


FIGURE 6.18. PCR was applied to two simulated data sets. Left: Simulated data from Figure 6.8. Right: Simulated data from Figure 6.9.

We considered $p = 45$ predictors and $n = 50$ observations. The left side of the figure shows a response Y depending on all variables, the right side only on 2 variables. In these two situations shrinkage methods performs better than Principal Components Regression.

6.67

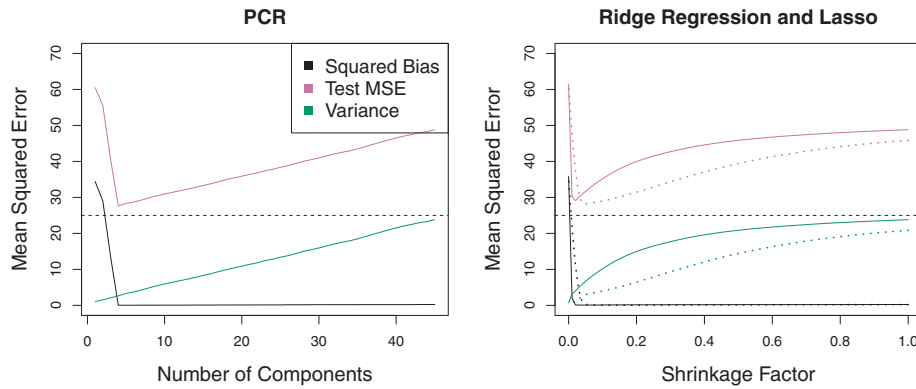


FIGURE 6.19. PCR, ridge regression, and the lasso were applied to a simulated data set in which the first five principal components of X contain all the information about the response Y . In each panel, the irreducible error $\text{Var}(\epsilon)$ is shown as a horizontal dashed line. Left: Results for PCR. Right: Results for lasso (solid) and ridge regression (dotted). The x-axis displays the shrinkage factor of the coefficient estimates, defined as the ℓ_2 norm of the shrunken coefficient estimates divided by the ℓ_2 norm of the least squares estimate.

6.68

Remarks

- Since each principal component is linear combination of all variables $X_1, \dots, X_p$, Principal Components Regression is not a *feature selection* method.
- Principal components should be obtained starting from standardized variables, that is by using the correlation matrix, not the covariance matrix.

6.69

Example 1 (Credit data Analysis). By having a look at the predictors correlation matrix we can expect that there exist one principal component linked to the variables limit and rating, that are collinear, and other 9 principal components, linked to the other predictors.

6.70

```
# data reading
credit <- read.table("Credit.csv", sep = ",", header = T)
# the first column contains rownames
credit <- credit[, -1]
# number of subjects
n <- nrow(credit)
# check for not available data
max(is.na(credit))
## [1] 0
y <- credit$Balance
x <- model.matrix(Balance ~ ., data = credit)[, -1]
```

6.71

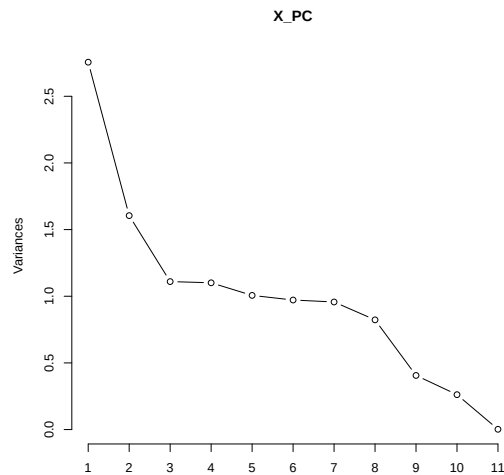
```
round(cor(x), 2)
##
## Income Limit Rating Cards Age Education GenderFemale
## Income 1.00 0.79 0.79 -0.02 0.18 -0.03 -0.01
## Limit 0.79 1.00 1.00 0.01 0.10 -0.02 0.01
## Rating 0.79 1.00 1.00 0.05 0.10 -0.03 0.01
## Cards -0.02 0.01 0.05 1.00 0.04 -0.05 -0.02
## Age 0.18 0.10 0.10 0.04 1.00 0.00 0.00
## Education -0.03 -0.02 -0.03 -0.05 0.00 1.00 -0.01
## GenderFemale -0.01 0.01 0.01 -0.02 0.00 -0.01 1.00
## StudentYes 0.02 -0.01 0.00 -0.03 -0.03 0.07 0.06
## MarriedYes 0.04 0.03 0.04 -0.01 -0.07 0.05 0.01
## EthnicityAsian -0.02 -0.03 -0.04 0.01 -0.06 0.03 0.03
## EthnicityCaucasian -0.02 0.00 0.00 -0.01 0.00 -0.04 -0.01
##
## StudentYes MarriedYes EthnicityAsian EthnicityCaucasian
## Income 0.02 0.04 -0.02 -0.02
## Limit -0.01 0.03 -0.03 0.00
## Rating 0.00 0.04 -0.04 0.00
## Cards -0.03 -0.01 0.01 -0.01
## Age -0.03 -0.07 -0.06 0.00
## Education 0.07 0.05 0.03 -0.04
## GenderFemale 0.06 0.01 0.03 -0.01
## StudentYes 1.00 -0.08 0.05 -0.05
## MarriedYes -0.08 1.00 0.09 0.01
## EthnicityAsian 0.05 0.09 1.00 -0.58
## EthnicityCaucasian -0.05 0.01 -0.58 1.00
```

6.72

```
# principal components extraction
X_PC <- prcomp(x, scale. = TRUE)
summary(X_PC)
## Importance of components:
##
## PC1 PC2 PC3 PC4 PC5 PC6
## Standard deviation 1.6601 1.2669 1.0536 1.0493 1.0032 0.98577
## Proportion of Variance 0.2505 0.1459 0.1009 0.1001 0.0915 0.08834
## Cumulative Proportion 0.2505 0.3964 0.4973 0.5974 0.6889 0.77727
##
## PC7 PC8 PC9 PC10 PC11
## Standard deviation 0.97831 0.90715 0.63723 0.51174 0.04618
## Proportion of Variance 0.08701 0.07481 0.03691 0.02381 0.00019
## Cumulative Proportion 0.86427 0.93908 0.97600 0.99981 1.00000
```

6.73

```
# screeplot
screeplot(X_PC, type = "lines", npcs = 11)
```



6.74

What is important is not how well principal components explain predictors variability, but how well do principal components explain the outcome. Therefore, the number of principal components is chosen by the best cross validation fit of MSE_{test} .

The function `pls::pcr` can be used to fit a Principal Components Regression.

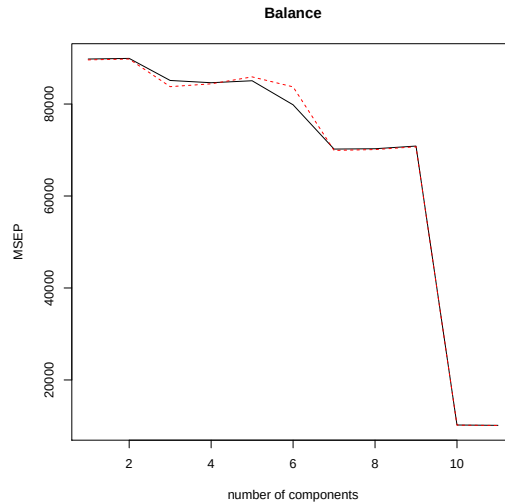
6.75

```
library(pls)
## ## Attaching package: 'pls'
## The following object is masked from 'package:stats': ## ##
loadings
pcr.fit <- pcr(Balance ~ ., data = credit, scale = TRUE, validation = "CV")
summary(pcr.fit)

## Data: X dimension: 400 11
## Y dimension: 400 1
## Fit method: svdpc
## Number of components considered: 11
##
## VALIDATION: RMSEP
## Cross-validated using 10 random segments.
##      (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps
## CV           460.3    299.6    299.9    291.8    290.9    291.7
## adjCV         460.3    299.3    299.6    289.4    290.5    293.1
##      6 comps  7 comps  8 comps  9 comps 10 comps 11 comps
## CV           282.5    265.0    265.1    266.2    101.0    100.7
## adjCV         289.4    264.4    264.8    265.9    100.8    100.5
##
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## X           25.05    39.64    49.73    59.74    68.89    77.73
## Balance      58.07    58.37    60.78    60.90    61.46    63.11
##      7 comps  8 comps  9 comps 10 comps 11 comps
## X           86.43    93.91    97.60    99.98    100.00
## Balance      68.70    68.71    68.72    95.47    95.51
# Root MSE is reported
```

6.76


```
# Plot with 10fold-CV MSE estimation
validationplot(pcr.fit, val.type = "MSEP", intercept = FALSE)
```



6.77

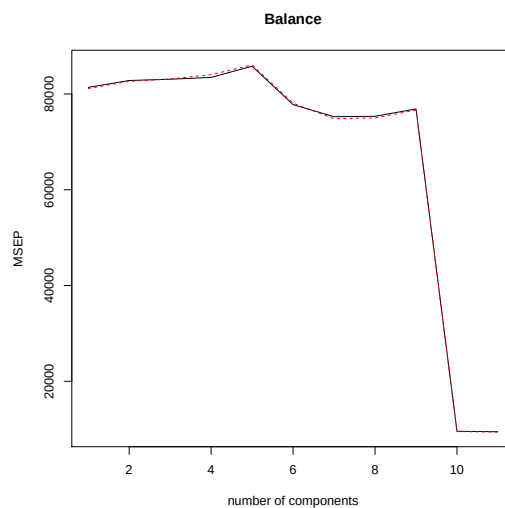
We estimate MSE_{test} by means of a 10-fold cross validation; the procedure is implemented in the function `pcr` by setting the argument `validation="CV"`.

6.78

```
# test MSE performance evaluation
set.seed(1234)
train <- sample(n, n/2)
pcr.fit <- pcr(Balance ~ ., data = credit, subset = train, scale = TRUE,
               validation = "CV")
```

6.79

```
# Plot with 10fold-CV MSE estimation
validationplot(pcr.fit, val.type = "MSEP", intercept = FALSE)
```



6.80

The selected model is equivalent to the one original estimated with 10 components (we re-fit it on the whole data set).

```
pcr.pred <- predict(pcr.fit, x[-train, ], ncomp = 10)
mean((pcr.pred - y[-train])^2)
## [1] 11273.3
```

```

pcr.fit <- pcr(Balance ~ ., data = credit, scale = TRUE, validation = "CV")
# it is possible to extract loadings and coefficients
loadings(pcr.fit)

##
## Loadings:
##          Comp 1 Comp 2 Comp 3 Comp 4 Comp 5 Comp 6 Comp 7
## Income          -0.542
## Limit            -0.586
## Rating           -0.587
## Cards                0.479 -0.272          -0.283  0.771
## Age              -0.123          0.107 -0.479 -0.295 -0.584 -0.359
## Education         -0.475  0.199 -0.583 -0.402  0.216
## GenderFemale      -0.334          0.746 -0.514 -0.102
## StudentYes        0.125 -0.619 -0.296          0.202  0.428
## MarriedYes         0.126  0.739          -0.324  0.136
## EthnicityAsian     0.697  0.106
## EthnicityCaucasian -0.687 -0.100  0.134          0.103
##          Comp 8 Comp 9 Comp 10 Comp 11
## Income                0.836
## Limit                -0.379  0.705
## Rating               -0.374 -0.708
## Cards              -0.109
## Age                0.413          -0.103
## Education          -0.418
## GenderFemale       -0.227
## StudentYes         0.534
## MarriedYes         0.537  0.119
## EthnicityAsian     -0.707
## EthnicityCaucasian -0.695
##
##          Comp 1 Comp 2 Comp 3 Comp 4 Comp 5 Comp 6 Comp 7
## SS loadings    1.000  1.000  1.000  1.000  1.000  1.000  1.000
## Proportion Var 0.091  0.091  0.091  0.091  0.091  0.091  0.091
## Cumulative Var 0.091  0.182  0.273  0.364  0.455  0.545  0.636
##          Comp 8 Comp 9 Comp 10 Comp 11
## SS loadings    1.000  1.000  1.000  1.000
## Proportion Var 0.091  0.091  0.091  0.091
## Cumulative Var 0.727  0.818  0.909  1.000

```

6.81

```

coef(pcr.fit, 10)

## , , 10 comps
##
##          Balance
## Income      -275.334437
## Limit       308.685448
## Rating      308.331638
## Cards       18.588390
## Age        -10.700222
## Education   -2.758126
## GenderFemale -5.354857
## StudentYes  127.056873
## MarriedYes  -5.131238
## EthnicityAsian  8.004166
## EthnicityCaucasian  5.143306
coef.pcr <- coef(pcr.fit, 1:11)[1:11, , 1:11]

```

6.82

```

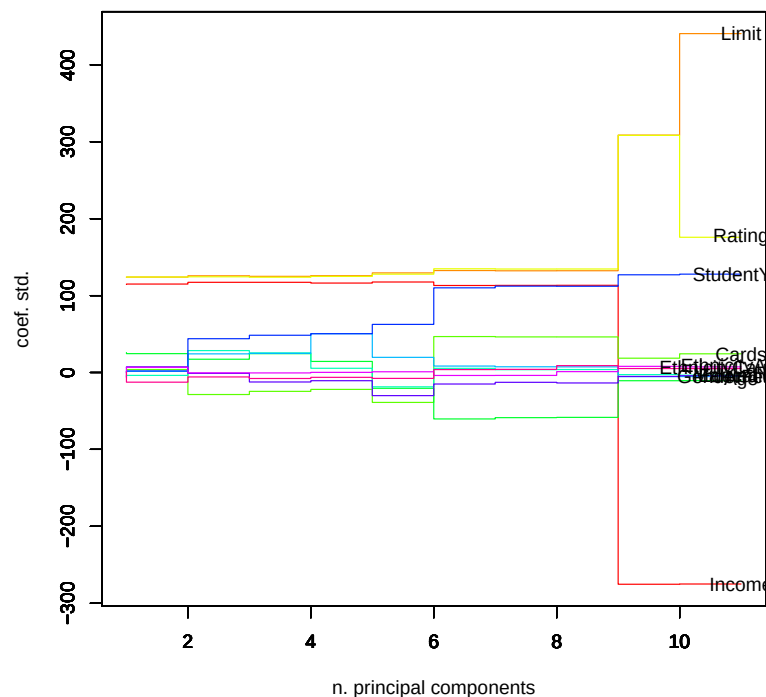
# plot coefficients for solutions with different number of
# principal components
col <- rainbow(11)

```

```

plot(0, 0, type = "n", xlim = c(1, 11), ylim = range(coef.pcr),
     xlab = "n. principal components", ylab = "coef. std.")
sapply(1:11, function(i) {
  par(new = TRUE)
  plot(1:11, coef.pcr[i, ], type = "S", xlim = c(1, 11), ylim = range(coef.pcr),
       xlab = "", ylab = "", col = col[i])
})
sapply(1:11, function(i) text(11, coef.pcr[i, 11], rownames(coef.pcr)[i]))

```



6.84

3 High Dimensional Data

Most traditional statistical techniques for regression and classification are intended for the low-dimensional setting ($n \gg p$) in which n , the number of observations, is much greater than p , the number of features.

In the past 20 years, new technologies have changed the way that data are collected in fields as diverse as finance, marketing, and medicine. It is now commonplace to collect an almost unlimited number of feature measurements (p very large). While p can be extremely large, the number of observations n is often limited due to cost, sample availability, or other considerations. An example is as follows:

A marketing analyst interested in understanding people's online shopping patterns could treat as features all of the search terms entered by users of a search engine. This is sometimes known as the "bag-of-words" model.

The same researcher might have access to the search histories of only a few hundred or a few thousand search engine users who have consented to share their information with the researcher.

6.85

For a given user, each of the p search terms is scored present (0) or absent (1), creating a large binary feature vector. Then $n \approx 1,000$ and p is much larger.

Therefore, we have $p \gg n$.

When it does not hold $n \gg p$ there is risk of overfitting. The estimate of the variance σ^2 of the error in a linear model

$$s^2 = \frac{1}{n - p - 1} RSS$$

is problematic, like the use of all feature selection criteria based on it, for example C_p , AIC , BIC and the adjusted R^2 .

With OLS we have seen that the model is identifiable if $n \geq p$; while if $n = p$ we have $MSE_{training} = 0$.

Care has to be considered when applying statistical methods.

Every analysis should be based on sound theory that supports the existence of a meaningful construct describing the relationship between Y and the predictors, that can possibly be included in the model.

The following figure describes the behavior of a linear model analyzing the relationship between the response Y , of $n = 20$ subjects, and the set of predictors with dimension $1, 2, \dots, 20$. No predictor is correlated with Y .

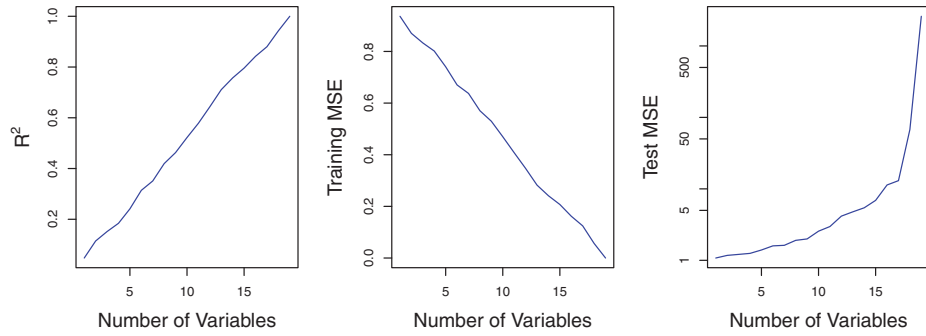


FIGURE 6.23. On a simulated example with $n = 20$ training observations, features that are completely unrelated to the outcome are added to the model. Left: The R^2 increases to 1 as more features are included. Center: The training set MSE decreases to 0 as more features are included. Right: The test set MSE increases as more features are included.

In these settings it should be more appropriate to apply methods characterized by lower flexibility than OLS, like the Forward Stepwise Selection, the ridge regression, the lasso or the Principal Components Regression.

We consider an example with $n = 100$, $p = 20, 50, 2000$, but only 20 predictors are linked to Y .

Observe as MSE_{test} increases when p increases.

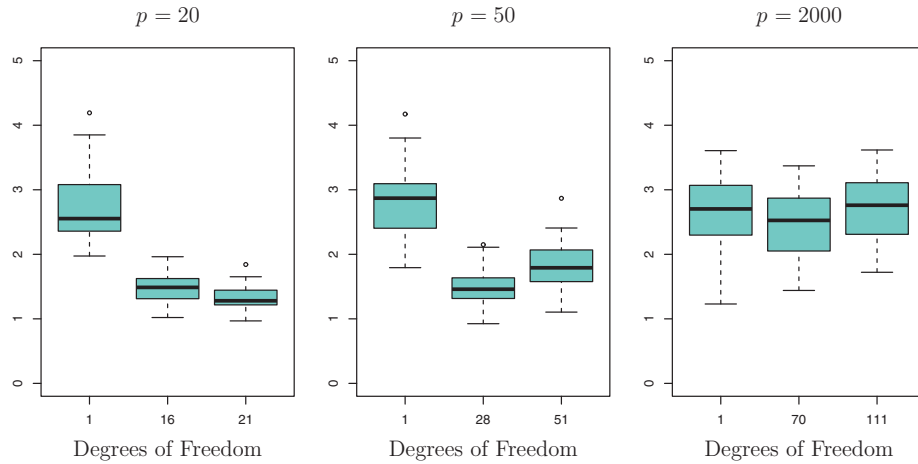


FIGURE 6.24. The lasso was performed with $n = 100$ observations and three values of p , the number of features. Of the p features, 20 were associated with the response. The boxplots show the test MSEs that result using three different values of the tuning parameter λ in (6.7). For ease of interpretation, rather than reporting λ , the degrees of freedom are reported; for the lasso this turns out to be simply the number of estimated non-zero coefficients. When $p = 20$, the lowest test MSE was obtained with the smallest amount of regularization. When $p = 50$, the lowest test MSE was achieved when there is a substantial amount of regularization. When $p = 2,000$ the lasso performed poorly regardless of the amount of regularization, due to the fact that only 20 of the 2,000 features truly are associated with the outcome.

6.91

- Regularization/shrinkage methods have an important role with reference to high dimensional problems.
- It is important to select accurately the tuning parameter λ with reference to the model predictive ability.
- The MSE_{test} can increase when p increases (noise variables, that are not linked with Y , can be inserted in the model – curse of dimensionality)
- The MSE_{test} will not increase when p increases, only if the added variables are actually linked with Y , and give additional information with respect to variables that are already in the model.

6.92

Curse of dimensionality

Usually,

- including new variables that *are linked* to the response Y increases the predictive ability of the model;
- including (noise) variables that *are not linked* to the response Y worsens the predictive ability of the model and increases the MSE_{test} .

This happens since the estimation method will try to find a relationship between Y and the new variables giving rise to overfitting, in presence of noise variables.

6.93

In big data analyses, a large number of predictors is useful only when they are truly associated with the response, that is relevant in explaining Y .

Even when all the $p \gg 1$ variables are relevant in explaining Y , it is important to evaluate if the bias reduction in fitting f with $\hat{f}$ justifies the increase of the variability of $\hat{f}$.

6.94

3.1 Interpreting Results in High Dimensions

In the high-dimensional setting, the multicollinearity problem is extreme:

- any variable in the model can be written as a linear combination of all of the other variables in the model.
- we can never know exactly which variables (if any) truly are predictive of the outcome Y , and we can never identify the *best* coefficients for use in the regression.
- At most, we can hope to assign large regression coefficients to variables that are correlated with the variables that truly are predictive of the outcome Y .

6.95

For instance, suppose that we are trying to predict blood pressure on the basis of half a million SNPs, and that forward stepwise selection indicates that 17 of those SNPs lead to a good predictive model on the training data. It would be incorrect to conclude that these 17 SNPs predict blood pressure more effectively than the other SNPs not included in the model. There are likely to be many sets of 17 SNPs that would predict blood pressure just as well as the selected model. If we were to obtain an independent data set and perform forward stepwise selection on that data set, we would likely obtain a model containing a different, and perhaps even non-overlapping, set of SNPs. This does not detract from the value of the model obtained—for instance, the model might turn out to be very effective in predicting blood pressure on an independent set of patients, and might be clinically useful for physicians. But we must be careful not to overstate the results obtained, and to make it clear that what we have identified is simply *one of many possible models* for predicting blood pressure, and that it must be further validated on independent data sets.

6.96

What we have identified is simply one of many possible models for predicting Y .

It must always be further validated on independent data sets.

It is also important to be particularly careful in reporting errors and measures of model fit in the high-dimensional setting.

Only evaluation measures obtained on independent data set have to be considered, that is compute the MSE_{test} or, in any case, estimate MSE_{test} by using cross validation methods.

6.97